

Day 28: Model Interpretability with SHAP

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 5, Day 1

Weeks 3-4 built and compared models, judged by aggregate metrics and global feature importance. Week 5 opens with the question stakeholders actually ask: not 'which features matter overall,' but 'why did the model predict this for this specific person.' Each topic below covers what it is, why it works, a real-world scenario, the exact implementation, a line-by-line breakdown, and when to reach for it versus when not to.

1. Why Global Feature Importance Isn't Enough

Topic specification: Gain-based feature importance (`classifier.feature_importances_`, covered Day 23/25) ranks features by how much they reduced loss across the whole trained model. It is a single global ranking — identical for every row the model will ever score.

Explanation: Gain importance answers 'which features did this model rely on most, on average.' It cannot answer 'why did this one row get this one prediction' — two rows can be driven by completely different features and gain importance has no way to show that.

Real-world scenario: A bank's loan-approval model gets gain importance showing Income and CreditScore as the top two features overall. A denied applicant asks why they specifically were denied — but their denial might actually be driven mostly by DebtToIncomeRatio, a feature that ranks lower globally but dominated their individual case. Regulators require the per-applicant answer, not the global one.

Implementation:

```
classifier = pipeline.named_steps["classifier"]
preprocessor = pipeline.named_steps["preprocessor"]
feature_names = preprocessor.get_feature_names_out()

gain_importance = pd.Series(
    classifier.feature_importances_, index=feature_names
).sort_values(ascending=False)
print(gain_importance)
```

Line-by-line explanation

- `named_steps["classifier"] / named_steps["preprocessor"]` — pull the fitted XGBoost model and ColumnTransformer out of the pipeline by their step names.
- `get_feature_names_out()` — returns the expanded column names after one-hot encoding, needed to label the plain numpy array `feature_importances_` returns.
- `pd.Series(...).sort_values(ascending=False)` — turns the raw importance array into a ranked, human-readable table, highest-impact feature first.
- This entire computation is a byproduct of training — no extra pass over the data is needed, which is exactly why it's cheap but limited to a single global view.

Why choose this? Gain importance is free — it's already computed during training, requires no extra library, and gives a fast sanity check on whether the model is relying on sensible features overall.

Why NOT this (tradeoffs)? It cannot explain any individual prediction, cannot show the direction of a feature's effect (did high Fare push survival up or down?), and can disagree with what actually drove any particular row's outcome — exactly the gap SHAP closes.

2. SHAP Values — Additive, Game-Theoretic Feature Attribution

Topic specification: SHAP (SHapley Additive exPlanations) values are built on Shapley values from cooperative game theory: treat each feature as a 'player' contributing to one prediction, and split credit for that prediction fairly among them, based on every possible order features could be 'added' to the model.

Explanation: The result is additive by construction: a base value (the model's average output across all training data) plus every feature's signed SHAP contribution sums exactly to that one row's prediction. No other feature-attribution method guarantees this exact additivity.

Real-world scenario: A company splitting bonus credit among three employees who jointly shipped a successful project considers every possible order they could have joined the effort in, and averages each person's marginal contribution across all those orderings — the same fairness principle SHAP applies to features instead of people.

Implementation:

```
import shap

explainer = shap.TreeExplainer(classifier)
shap_values = explainer.shap_values(X_row)
base_value = explainer.expected_value
```

Line-by-line explanation

- `shap.TreeExplainer(classifier)` — builds an explainer object around the already-fitted classifier; no retraining happens.
- `explainer.shap_values(X_row)` — returns one signed contribution value per feature, per row passed in.
- `explainer.expected_value` — the base value: what the model would output on average with no feature information, the starting point every explanation is built from.
- Every SHAP value is in the same units as the model's raw output (log-odds for XGBoost's binary classification), not directly in probability units — covered in Topic 4.

Why choose this? SHAP values are mathematically consistent and fair by construction — a feature's SHAP value never decreases if the model relies on it more, a guarantee simpler heuristics don't provide — and they explain individual rows, not just the model as a whole.

Why NOT this (tradeoffs)? Computing exact Shapley values is combinatorially expensive in general (every possible feature ordering); it's only fast here because `TreeExplainer` exploits tree structure

specifically. For non-tree models, SHAP falls back to slower approximation methods.

3. TreeExplainer for Tree Ensembles

Topic specification: TreeExplainer is a SHAP explainer variant built specifically for tree-based models — random forest, gradient boosting, XGBoost, LightGBM. It computes exact SHAP values by walking each tree's actual split structure, rather than approximating them by sampling.

Explanation: Because it uses the tree structure directly, TreeExplainer is both exact (not an approximation) and fast — a critical difference from KernelExplainer, SHAP's model-agnostic fallback, which perturbs input features randomly and is far slower at scale.

Real-world scenario: An e-commerce platform explaining millions of daily recommendation-ranking predictions from an XGBoost model needs explanations fast enough to log alongside every prediction in production. TreeExplainer's speed makes that practical; KernelExplainer's sampling-based approach would be far too slow to run at that volume.

Implementation:

```
# xgboost >= 2.0 serializes base_score in a bracketed format
# that crashes shap's model loader – patched before use:
import shap.explainers._tree as _shap_tree_module
_original_decode = _shap_tree_module.decode_ubjson_buffer

def _strip_base_score_brackets(obj):
    if isinstance(obj, dict):
        for k, v in obj.items():
            if k == "base_score" and isinstance(v, str) and v.startswith("["):
                obj[k] = v.strip("[")
            else:
                _strip_base_score_brackets(v)
    elif isinstance(obj, list):
        for item in obj:
            _strip_base_score_brackets(item)

def _patched_decode(fd):
    jmodel = _original_decode(fd)
    _strip_base_score_brackets(jmodel)
    return jmodel

_shap_tree_module.decode_ubjson_buffer = _patched_decode

explainer = shap.TreeExplainer(classifier)
```

Line-by-line explanation

- The comment block documents a real, verified compatibility issue: XGBoost 2.0+ stores `base_score` as a bracketed string like `"[0.313]"`, and SHAP's loader calls `float()` directly on it, which raises a `ValueError`.

- `_strip_base_score_brackets` — recursively walks the decoded model dictionary and strips the brackets from any `base_score` string it finds, in place.
- `_patched_decode` — wraps SHAP's original UBJSON decoder, running the fix immediately after decoding, before SHAP tries to parse `base_score` as a float.
- `_shap_tree_module.decode_ubjson_buffer = _patched_decode` — monkey-patches the fixed function into SHAP's internals for the rest of the script's run; it does not modify the installed shap package itself.
- `shap.TreeExplainer(classifier)` — now succeeds, because it uses the patched decoder internally when it loads the XGBoost model.

Why choose this? For any tree-based model, TreeExplainer is strictly better than the model-agnostic alternative — exact instead of approximate, and dramatically faster, with no real downside.

Why NOT this (tradeoffs)? It only works for tree ensembles. A neural network, SVM, or linear model needs a different SHAP explainer (DeepExplainer, LinearExplainer, or the slower KernelExplainer as a general fallback) — TreeExplainer cannot be pointed at an arbitrary model type.

4. Explaining One Individual Prediction

Topic specification: For a single row, SHAP produces one signed contribution per feature. Summing the base value and every contribution reconstructs that row's raw model output exactly; applying a sigmoid converts that log-odds sum into the same probability `predict_proba` would report.

Explanation: XGBoost's binary classifier outputs log-odds internally before the final sigmoid conversion to a 0-1 probability. SHAP values for XGBoost are additive in that same log-odds space, not directly in probability space — a common source of confusion if the sigmoid step is skipped.

Real-world scenario: An insurance company's claim-risk model flags a specific customer's claim as high risk. The customer disputes it, and a claims adjuster needs to show — not just tell — which specific factors (recent claim history, vehicle age, location) drove that one score, in a way that reconstructs the exact number the model produced.

Implementation:

```
row = X_test_transformed[row_index:row_index + 1]
shap_values = explainer.shap_values(row)
base_value = explainer.expected_value

contributions = pd.Series(shap_values[0], index=feature_names)
margin_sum = base_value + contributions.sum()
reconstructed_prob = 1 / (1 + np.exp(-margin_sum))

print(reconstructed_prob)
print(pipeline.named_steps["classifier"].predict_proba(row)[0][1])
```

Line-by-line explanation

- `row = X_test_transformed[row_index:row_index + 1]` — slices out exactly one already-preprocessed row, keeping it 2D as scikit-learn/XGBoost expect.
- `shap_values[0]` — the array of per-feature contributions for that single row.
- `margin_sum = base_value + contributions.sum()` — reconstructs the raw log-odds output by adding every feature's signed contribution to the base value.
- `1 / (1 + np.exp(-margin_sum))` — the sigmoid function, converting log-odds back into a 0-1 probability.
- Verified in this lesson: this manually reconstructed probability (0.298) matched `predict_proba()`'s output (0.298) exactly, confirming the SHAP values are correctly additive.

Why choose this? Per-row explanations give individual accountability — the exact answer to 'why did this specific row get this specific prediction,' which is legally required in some regulated domains (credit, insurance, hiring) and simply good practice everywhere else.

Why NOT this (tradeoffs)? A single row's explanation says nothing about the model's behavior in general — it doesn't reveal systemic bias or overall reliability. Per-row and global explanations answer different questions and are both needed, not one instead of the other.

5. Global SHAP Importance vs Gain-Based Importance

Topic specification: Averaging the absolute SHAP value of each feature across every row produces a second global importance ranking — mean |SHAP value| — that can be compared directly against XGBoost's built-in gain-based ranking from Topic 1.

Explanation: Gain measures how much a feature reduced loss when splits used it during training. Mean |SHAP value| measures how much a feature actually moved individual predictions, averaged across every row it appears in — a subtly different question that can reorder features gain ranks similarly.

Real-world scenario: A model-governance team validating a newly trained credit-risk model before deployment checks that both importance methods roughly agree on the top drivers (income, credit history) — agreement builds confidence the model learned genuine signal rather than an artifact of how gain happens to be computed.

Implementation:

```
shap_values_all = explainer.shap_values(X_train_transformed)
mean_abs_shap = pd.Series(
    np.abs(shap_values_all).mean(axis=0), index=feature_names
).sort_values(ascending=False)

comparison_table = pd.DataFrame({
    "gain_rank": gain_importance.rank(ascending=False),
    "shap_rank": mean_abs_shap.reindex(gain_importance.index).rank(ascending=False),
})
print(comparison_table)
```

Line-by-line explanation

- `explainer.shap_values(X_train_transformed)` — computes SHAP values for every training row at once, not just one row as in Topic 4.
- `np.abs(shap_values_all).mean(axis=0)` — averages the absolute contribution of each feature across all rows, collapsing signed row-level detail into one global magnitude per feature.
- `.reindex(gain_importance.index)` — aligns the SHAP ranking to the same feature order as the gain ranking before comparing them side by side.
- Verified in this lesson: Sex and Pclass_3 ranked #1 and #2 under both methods, while Age and Fare swapped positions between the two rankings — the two methods mostly agree on what matters most, but not always on the exact order further down.

Why choose this? SHAP-based global importance accounts for feature interactions and is grounded in the same consistent, theoretically justified attribution used for individual predictions — one coherent framework for both the global and local view.

Why NOT this (tradeoffs)? It costs an extra computation pass over the data (running the explainer across every row), unlike gain importance which is free from training. For a quick sanity check with no need for row-level explanations later, gain importance alone may be sufficient.

6. Row-Level Comparison Across Contrasting Predictions

Topic specification: Pulling the SHAP explanation for the test set's highest-confidence and lowest-confidence predictions side by side shows whether the same features drive both, or whether different rows are dominated by entirely different factors.

Explanation: There is no guarantee that the globally top-ranked feature is what drove any particular row's prediction. Comparing two contrasting rows directly demonstrates this: each can have its own distinct top-3 contributors, invisible from any single global ranking.

Real-world scenario: A fraud-detection analyst reviews two transactions flagged by the same model — one at 99% fraud probability, one at 2%. SHAP shows the high-risk flag was driven mainly by transaction location and amount, while the low-risk score was driven mainly by account tenure — two different stories the analyst needs to review differently, not a single blanket explanation.

Implementation:

```
probs = classifier.predict_proba(X_test_transformed)[: , 1]
highest_idx = int(np.argmax(probs))
lowest_idx = int(np.argmin(probs))

for label, idx in [("highest", highest_idx), ("lowest", lowest_idx)]:
    row = X_test_transformed[idx:idx + 1]
    shap_values = explainer.shap_values(row)
    top3 = pd.Series(shap_values[0], index=feature_names) \
        .sort_values(key=abs, ascending=False).head(3)
    print(label, probs[idx], top3)
```

Line-by-line explanation

- `np.argmax(probs) / np.argmin(probs)` — find the row indices of the most and least confident survival predictions in the test set.
- `.sort_values(key=abs, ascending=False).head(3)` — ranks each row's own contributions by absolute size and keeps only the top 3, the features that mattered most for that specific row.
- Verified in this lesson: the highest-probability passenger ($p=0.958$) was driven most by Fare and Sex, while the lowest-probability passenger ($p=0.011$) was driven most by Fare, Sex, and Age — overlapping but not identical, and in opposite directions.

Why choose this? Case-by-case auditing surfaces different failure modes and different reasoning patterns across rows — essential for debugging a specific wrong prediction or building trust with an individual stakeholder questioning their result.

Why NOT this (tradeoffs)? Manually reviewing rows one at a time doesn't scale to auditing an entire deployed model for systemic bias or drift — that still requires the aggregate, global view from Topic 5 and Day 23/25's gain importance, used alongside row-level checks, not as a replacement for them.

Interview Preparation — Technical Questions

Q: What guarantee makes SHAP values 'additive,' and why does that matter?

A: The base value plus every feature's SHAP contribution sums exactly to the model's raw output for that row. This matters because it means the explanation is a complete, exact decomposition of the prediction — not an approximation or a partial account of what drove it.

Q: Why is TreeExplainer faster than KernelExplainer for an XGBoost model?

A: TreeExplainer computes exact SHAP values by walking the actual tree structure the model already contains. KernelExplainer is model-agnostic — it approximates SHAP values by repeatedly perturbing input features and observing output changes, which is far slower and only approximate.

Q: Why are SHAP values for an XGBoost binary classifier in log-odds space rather than probability space by default?

A: XGBoost's internal model output before the final sigmoid activation is a log-odds (margin) value. SHAP explains that raw model output additively; converting to a 0-1 probability requires explicitly applying the sigmoid function to the summed base value and contributions.

Q: What's the practical difference between gain-based importance and mean |SHAP value|?

A: Gain measures how much a feature's splits reduced loss during training, averaged across trees — a byproduct of the training process itself. Mean |SHAP value| measures how much a feature actually moved individual predictions, averaged across rows — computed in a separate pass after training, using the consistent Shapley attribution framework.

Q: Why did this lesson need to monkey-patch shap's UBJSON decoder?

A: XGBoost 2.0+ serializes the model's `base_score` parameter as a bracketed string (e.g. "[0.313]") to support multi-target models. SHAP's TreeExplainer calls `float()` directly on that string, which raises a `ValueError`. The patch strips the brackets from the decoded model dictionary before SHAP parses `base_score`, without modifying the installed shap package.

Q: Can TreeExplainer be used on a linear model or a neural network?

A: No — TreeExplainer relies on walking the internal tree-split structure of tree-based models specifically. A linear model would use LinearExplainer, a neural network would use DeepExplainer or GradientExplainer, and KernelExplainer works as a slower, model-agnostic fallback for any model type.

Q: Why can a feature rank highly in gain-based importance but not dominate any single row's SHAP explanation?

A: Gain-based importance is an average across the entire model's training process; a feature can accumulate a lot of total gain by contributing modestly across many splits and many rows without ever being the single largest contributor to any one row's specific prediction.

Q: What does `explainer.expected_value` represent?

A: The base value — the model's average raw output across the training (or background) data, with no feature information yet applied. It's the starting point every individual prediction's explanation is built from by adding that row's feature contributions.

Interview Preparation — Theoretical Questions

Q: Why is it insufficient, in a regulated setting like lending, to justify a model's decision using only global feature importance?

A: Global importance describes average behavior across the whole population the model was trained on, not the specific reasoning behind any individual decision. Regulations (e.g. adverse action notices in credit lending) typically require an explanation for the specific applicant's specific outcome, which only a per-row method like SHAP can provide.

Q: Conceptually, why does the Shapley value framework consider every possible ordering of features being 'added' to the model?

A: A feature's marginal contribution can depend on which other features are already present — it isn't necessarily a fixed, order-independent quantity. Averaging the marginal contribution across every possible ordering is what makes the resulting attribution fair and consistent regardless of any arbitrary choice of feature order, which is the core fairness property borrowed from cooperative game theory.

Q: Why might mean |SHAP value| and gain-based importance disagree on some features even though both are computed from the same trained model?

A: Gain rewards a feature for splits that reduced loss regardless of how many rows those splits actually affected. Mean |SHAP value| rewards a feature for how much it actually shifted predictions across all rows. A feature could produce one very loss-reducing split that only fires rarely (high gain, lower mean SHAP) or many modest splits that fire constantly (lower gain per split, higher mean SHAP) — the two metrics are answering related but distinct questions.

Q: Why is per-row SHAP analysis alone insufficient for auditing a deployed model for systemic bias?

A: Reviewing individual rows one at a time cannot scale to detecting a pattern that only emerges in aggregate — for example, a feature systematically pushing predictions against a protected group across thousands of rows, each individually small enough not to look alarming. Systemic bias detection requires aggregating SHAP values across many rows (as in Topic 5) or a dedicated fairness audit, not row-by-row inspection alone.

Q: What does it mean that SHAP values are 'consistent,' and why is that property useful when comparing two versions of a model?

A: Consistency means a feature's SHAP value cannot decrease if the model comes to rely on it more (holding all else equal). This makes SHAP values reliable for comparing feature importance across model versions or configurations — a heuristic without this guarantee could show a feature's importance dropping even when the model actually relies on it more than before, which would be misleading during model iteration.

Q: Why does explaining one prediction in log-odds space, rather than probability space, actually make the individual feature contributions more interpretable in aggregate, even though probability is more intuitive to read?

A: Log-odds is the space in which the model's contributions are genuinely additive — each feature's effect adds up linearly regardless of the other features' values. Probability is a nonlinear (sigmoid) transformation of that sum, so feature contributions computed directly in probability space would not

sum consistently across different rows, breaking the additivity guarantee that makes SHAP values comparable and combinable in the first place.

Q: Why does encountering and fixing a real library version-compatibility bug (as with the `shap/xgboost base_score` issue) matter as much as understanding the underlying theory?

A: Production machine learning work constantly runs into exactly this kind of friction — library versions drift apart faster than their documentation updates, and a correct theoretical understanding of SHAP doesn't by itself resolve a `ValueError` blocking every downstream analysis. Debugging this kind of issue systematically (reading the traceback, isolating where the failure occurs, patching narrowly rather than downgrading blindly) is a practical skill interviewers explicitly probe for, separate from conceptual knowledge.

Q: How does SHAP's local, per-row explanation connect back to the ensembling work from Week 4?

A: A stacking or voting ensemble's final prediction is itself a combination of multiple base models' outputs — SHAP can be applied to each base model independently to see which features drove each one's contribution to a disputed row, revealing not just what the ensemble predicted but which base model's reasoning actually dominated that specific outcome.

Key Takeaway and What's Next

Key takeaway: Global feature importance (Day 23/25) and per-row SHAP explanations (today) answer different questions and are both necessary — global importance for a quick sanity check and systemic auditing, SHAP for individual accountability and debugging specific predictions. Neither replaces the other.

What typically follows model interpretability

- `Visual SHAP tools` — `force plots` and `summary/beeswarm plots` (`shap.plots`) turn today's printed tables into the visual explanations most stakeholders and dashboards actually expect.
- `Partial dependence and ICE plots` — show how a prediction changes as one feature varies while others are held fixed, complementing SHAP's per-row attribution with a per-feature sensitivity view.
- `Model persistence and serving` — once a model's behavior is understood and trusted, saving it with `joblib` and serving it behind a minimal API turns this week's work into something a portfolio project can demonstrate end-to-end.
- `Hyperparameter search at scale` — `RandomizedSearchCV` or `Optuna`-style Bayesian search, for when a parameter grid grows too large for the `GridSearchCV` approach used through Week 4.